

Late delivery risk pipeline (Hệ thống phát hiện sớm rủi ro giao hàng trễ)

Mục lục

1. Giới thiệu dự án
 - 1.1 Tổng quan
 - 1.2 Mục tiêu
 - 1.3 Phạm vi
 - 1.4 Kết quả đạt được
 - 1.5 Ý nghĩa
2. Bài toán nghiệp vụ
 - 2.1 Bối cảnh
 - 2.2 Vấn đề
 - 2.3 Giải pháp
 - 2.4 Kết quả mong đợi
3. Kiến trúc hệ thống
 - 3.1 Kiến trúc tổng thể
 - 3.2 Nguồn dữ liệu (Data Source)
 - 3.3 Data Pipeline
 - 3.4 Lưu trữ dữ liệu (Data Storage)
 - 3.5 Trực quan hóa (Data Visualization)
4. Thiết kế Data Pipeline
 - 4.1 Luồng xử lý
 - 4.2 Full Load
 - 4.3 Incremental Load
 - 4.4 Kiến trúc Medallion (Bronze–Silver–Gold)

- 4.5 Quản lý metadata & watermark
 - 4.6 Điều phối workflow (Apache Airflow)
 - 5. Thiết kế lưu trữ & mô hình dữ liệu
 - 5.1 Bronze Layer
 - 5.2 Silver Layer
 - 5.3 Gold Layer
 - 5.4 Data Warehouse
 - 5.5 Star Schema
 - 6. Công nghệ sử dụng
 - 7. Kết quả & hướng phát triển
 - 7.1 Kết quả
 - 7.2 Hướng phát triển
-

Late Delivery Risk Pipeline (Hệ thống phát hiện sớm rủi ro giao hàng trễ)

1. Giới thiệu dự án

1.1 Tổng quan

Late Delivery Risk Pipeline là dự án Data Engineering mô phỏng quy trình xử lý dữ liệu trong một hệ thống thương mại điện tử. Dự án sử dụng **Brazilian E-Commerce Public Dataset by Olist**

https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce?select=olist_geolocation_dataset.csv

và dữ liệu phát sinh mới được tạo bằng **Faker** để xây dựng hệ thống dữ liệu hoàn chỉnh từ thu thập → lưu trữ → xử lý → trực quan hóa.

Dữ liệu được xử lý theo kiến trúc **Medallion** gồm 3 tầng **Bronze – Silver – Gold** trên **Databricks**. Quy trình ETL được tự động hóa bằng **Apache Airflow**, hỗ trợ:

- **Full Load** cho các bảng tham chiếu ít thay đổi
- **Incremental Load** theo **watermark** dựa trên *source timestamp* nhằm chỉ xử lý các bản ghi mới phát sinh ở mỗi lần chạy

Sau khi xử lý, dữ liệu tầng **Gold** được dùng để xây dựng dashboard trên **Streamlit**, phục vụ theo dõi hoạt động giao hàng và phát hiện sớm các đơn hàng có nguy cơ giao trễ.

1.2 Mục tiêu

- Xây dựng hệ thống Data Pipeline end-to-end.
- Thiết kế Data Lake theo kiến trúc Medallion trên Databricks.
- Tự động hóa ETL bằng Apache Airflow.
- Hỗ trợ Full Load và Incremental Load.
- Áp dụng watermark theo source timestamp để đồng bộ dữ liệu gia tăng.
- Kiểm soát chất lượng dữ liệu trong quá trình xử lý.
- Chuẩn hóa dữ liệu trước khi phân tích.
- Xây dựng các bảng tầng Gold phục vụ phân tích.
- Phát triển dashboard theo dõi tình trạng giao hàng bằng Streamlit.

1.3 Phạm vi

Dự án tập trung vào xử lý dữ liệu theo mô hình **Batch Processing**, kết hợp dữ liệu lịch sử và dữ liệu phát sinh mới.

Các thành phần chính:

- Thu thập dữ liệu từ **PostgreSQL**
- Sinh dữ liệu giao dịch mới bằng **Faker**
- Trích xuất dữ liệu theo cơ chế Full Load / Incremental Load
- Lưu trữ dữ liệu thô dạng **Parquet** tại Bronze Layer
- Chuyển đổi và làm sạch tại Silver Layer
- Xây dựng các bảng phân tích tại Gold Layer
- Trực quan hóa dữ liệu bằng **Streamlit Dashboard**

Lưu ý: Dự án **không** xây dựng mô hình Machine Learning dự đoán giao hàng trễ; mục tiêu chính là xây dựng nền tảng dữ liệu phục vụ phân tích và mở rộng về sau.

1.4 Kết quả đạt được

- Quản lý dữ liệu giao dịch và metadata trên PostgreSQL.
- Sinh dữ liệu giao dịch mới để mô phỏng dữ liệu phát sinh.
- Đồng bộ dữ liệu mới tự động bằng Incremental Loading.
- Quản lý watermark theo từng bảng dữ liệu.
- Lưu trữ dữ liệu theo kiến trúc Medallion trên Databricks.
- Làm sạch và chuẩn hóa dữ liệu trước phân tích.
- Xây dựng dữ liệu tổng hợp phục vụ phát hiện rủi ro giao hàng trễ.
- Hiển thị dữ liệu trên dashboard Streamlit.
- Tự động hóa toàn bộ quy trình bằng Apache Airflow.

1.5 Ý nghĩa

Trong hệ thống thương mại điện tử, việc phát hiện sớm các đơn hàng có nguy cơ giao trễ giúp doanh nghiệp chủ động theo dõi tiến độ, đánh giá hiệu suất vận hành của người bán/đơn vị vận chuyển và nâng cao trải nghiệm khách hàng.

Giải pháp **Incremental Load + watermark** giúp:

- Giảm thời gian xử lý và tiết kiệm tài nguyên so với việc nạp lại toàn bộ dữ liệu
- Đảm bảo dữ liệu phân tích luôn cập nhật theo từng lần chạy

2. Bài toán nghiệp vụ

2.1 Bối cảnh

Olist là nền tảng thương mại điện tử tại Brazil, kết nối khách hàng với nhiều nhà bán hàng trên toàn quốc. Thời gian giao hàng chịu ảnh hưởng bởi nhiều yếu tố (tốc độ xử lý của người bán, đơn vị vận chuyển, khoảng cách địa lý...), nên một số đơn hàng có thể giao muộn hơn dự kiến.

Dữ liệu giao dịch được lưu trữ đầy đủ nhưng phân tán giữa nhiều bảng, gây khó khăn trong theo dõi và phân tích.

2.2 Vấn đề

Hệ thống hiện tại chưa đáp ứng tốt các nhu cầu:

- Chưa có pipeline tự động thu thập và xử lý dữ liệu.
- Chưa có cơ chế Incremental Loading để chỉ xử lý dữ liệu mới.
- Dữ liệu chưa được chuẩn hóa phục vụ phân tích.
- Chưa có mô hình dữ liệu tối ưu cho BI.
- Chưa có dashboard theo dõi rủi ro giao hàng trễ.

2.3 Giải pháp

Xây dựng Data Pipeline end-to-end nhằm:

- Thu thập dữ liệu từ PostgreSQL.
- Hỗ trợ Full Load và Incremental Load theo source timestamp.
- Tổ chức dữ liệu theo Medallion (Bronze/Silver/Gold).
- Chuẩn hóa và biến đổi dữ liệu trước phân tích.
- Xây dựng Data Warehouse theo Galaxy **Schema**.
- Cung cấp dữ liệu cho dashboard theo dõi nguy cơ giao hàng trễ.

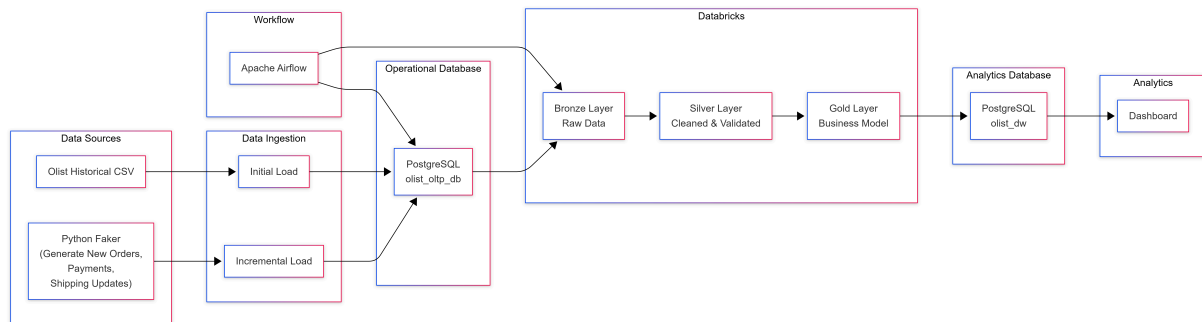
2.4 Kết quả mong đợi

- Tự động cập nhật dữ liệu theo lịch bằng Apache Airflow.
- Chỉ xử lý dữ liệu mới phát sinh, giảm thời gian ETL.
- Cung cấp dữ liệu chuẩn hóa và nhất quán.
- Hỗ trợ phân tích hiệu suất giao hàng và phát hiện sớm rủi ro giao trễ.
- Cung cấp dữ liệu ổn định cho dashboard và báo cáo.

3. Kiến trúc hệ thống

Hệ thống được xây dựng theo mô hình **Data Pipeline end-to-end** với kiến trúc phân tầng, tách biệt các bước thu thập, xử lý, lưu trữ và trực quan hóa. Dữ liệu được lấy từ PostgreSQL, xử lý bởi Airflow + Databricks và cung cấp cho Streamlit dashboard.

3.1 Kiến trúc tổng thể



Bốn thành phần chính:

- **Data Source**
- **Data Pipeline**
- **Data Storage**
- **Data Visualization**

3.2 Nguồn dữ liệu (Data Source)

Nguồn dữ liệu chính: **PostgreSQL** chứa dữ liệu giao dịch Olist và dữ liệu mô phỏng từ Faker.

Hai hình thức nạp dữ liệu:

- **Full Load:** áp dụng cho bảng ít thay đổi
- **Incremental Load:** áp dụng cho bảng phát sinh dữ liệu mới, dựa trên watermark theo source timestamp

3.3 Data Pipeline

Pipeline được xây dựng bằng Python và điều phối bởi Apache Airflow. Các bước chính:

1. Đọc cấu hình pipeline.
2. Xác định load type cho từng bảng.
3. Trích xuất dữ liệu từ PostgreSQL.
4. Ghi dữ liệu vào Bronze (Parquet).
5. Upload dữ liệu lên Databricks Volume.

6. Chạy xử lý Silver.
7. Chạy xử lý Gold.
8. Cập nhật metadata/watermark sau khi chạy thành công.

3.4 Lưu trữ dữ liệu (Data Storage)

Theo kiến trúc Medallion:

- **Bronze:** dữ liệu gốc
- **Silver:** làm sạch/chuẩn hóa
- **Gold:** dữ liệu tổng hợp phục vụ phân tích & dashboard

3.5 Trực quan hóa (Data Visualization)

Dữ liệu Gold được dùng để xây dựng dashboard bằng Streamlit nhằm theo dõi KPI và phát hiện sớm các đơn hàng có nguy cơ giao trễ.

4. Thiết kế Data Pipeline

4.1 Luồng xử lý

1. Đọc cấu hình các bảng cần xử lý.
2. Xác định Full Load hoặc Incremental Load.
3. Trích xuất dữ liệu từ PostgreSQL.
4. Ghi Parquet vào Bronze.
5. Upload lên Databricks Volume.
6. Biến đổi Bronze → Silver.
7. Xây dựng bảng Gold phục vụ phân tích.
8. Cập nhật metadata/watermark.

4.2 Full Load

Áp dụng cho bảng tham chiếu ít thay đổi. Mỗi lần chạy sẽ trích xuất toàn bộ dữ liệu và ghi vào Bronze.

Danh sách bảng Full Load:

- customers
- geolocation
- products
- sellers
- product_category_name_translation
- order_payments

4.3 Incremental Load

Áp dụng cho bảng phát sinh dữ liệu mới để giảm khối lượng xử lý. Pipeline chỉ đọc các bản ghi có timestamp lớn hơn watermark đã lưu trong metadata.

Bảng	Watermark (source timestamp)
orders	order_purchase_timestamp
order_items	order_purchase_timestamp (<i>thông qua bảng orders</i>)
order_reviews	review_creation_date

Sau mỗi lần chạy thành công, watermark mới sẽ được cập nhật để dùng cho lần tiếp theo.

4.4 Kiến trúc Medallion (Bronze–Silver–Gold)

- **Bronze:** lưu dữ liệu gốc từ PostgreSQL (Parquet), không thay đổi nội dung.
- **Silver:** chuẩn hóa kiểu dữ liệu, xử lý thiếu/không hợp lệ, loại trùng lặp, kiểm tra chất lượng.
- **Gold:** tổng hợp dữ liệu, tối ưu truy vấn cho BI/dashboard.

4.5 Quản lý metadata & watermark

Pipeline sử dụng 2 bảng metadata:

pipeline_metadata (cấu hình & trạng thái từng bảng):

- load_type
- watermark_column
- last_watermark

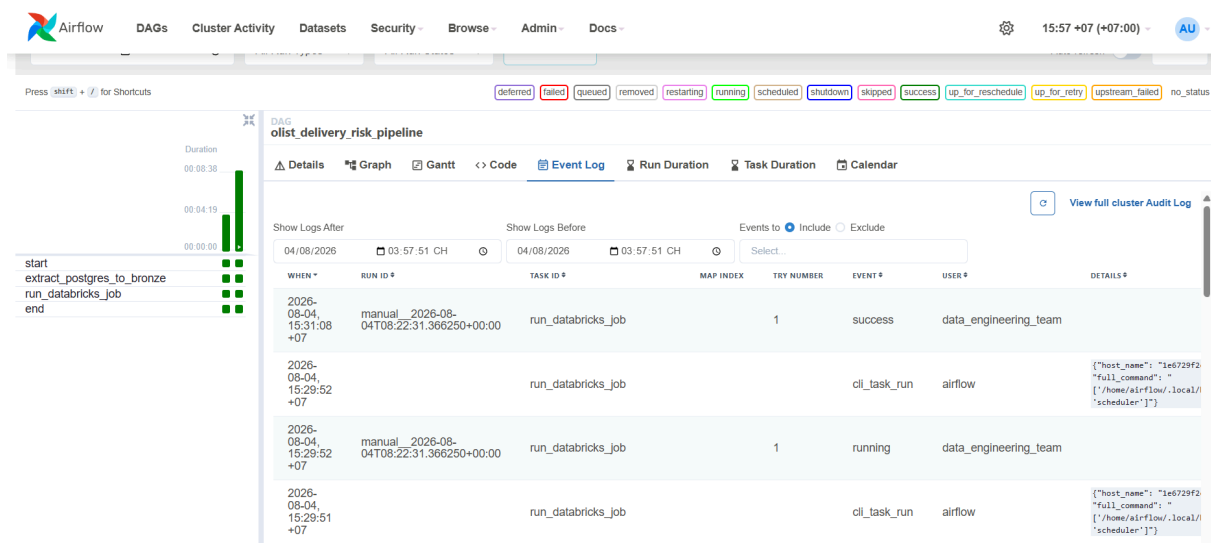
- last_batch_id
- last_run_time

pipeline_run_history (lịch sử chạy pipeline):

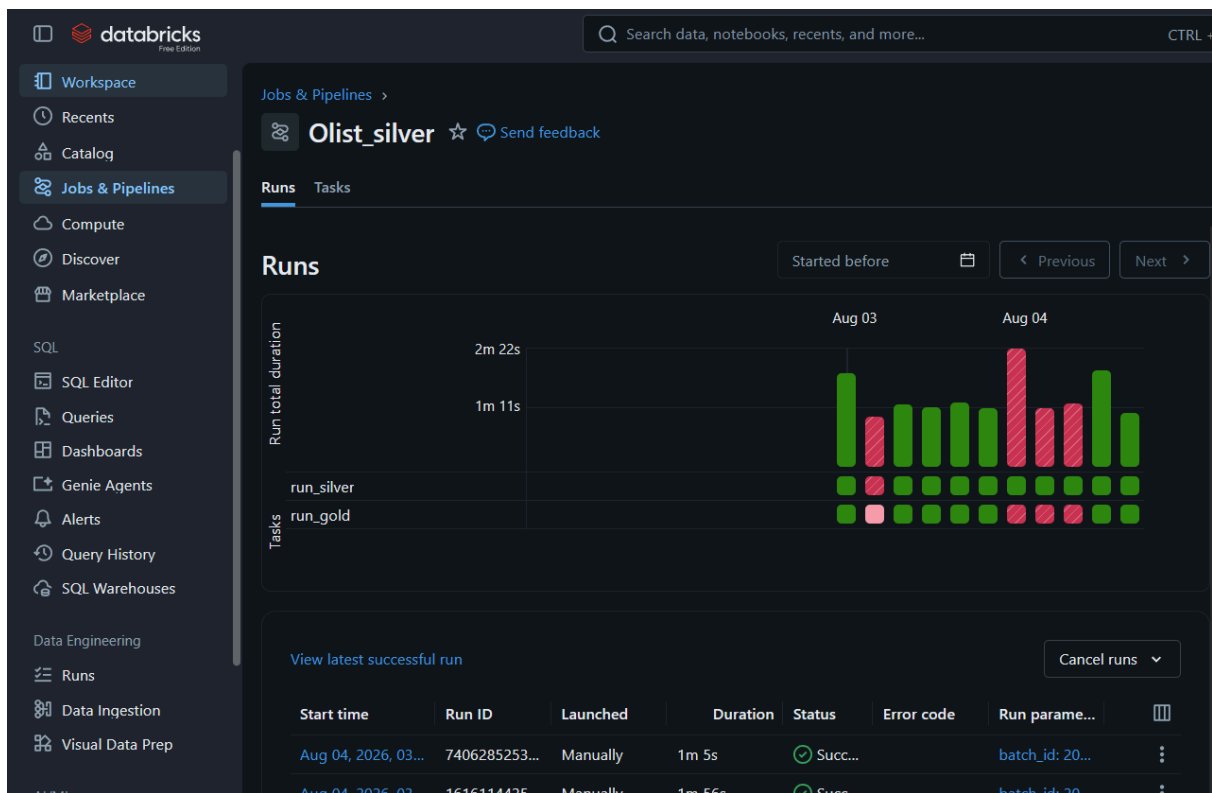
- batch_id
- table_name
- start_time
- end_time
- record_count
- status
- error_message

4.6 Điều phối workflow (Apache Airflow)

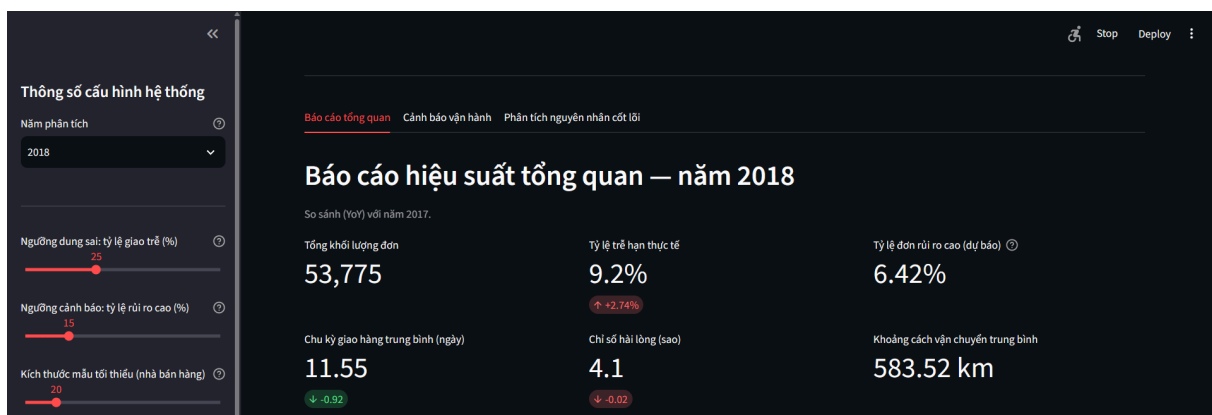
Mỗi lần DAG được kích hoạt, Airflow sẽ:

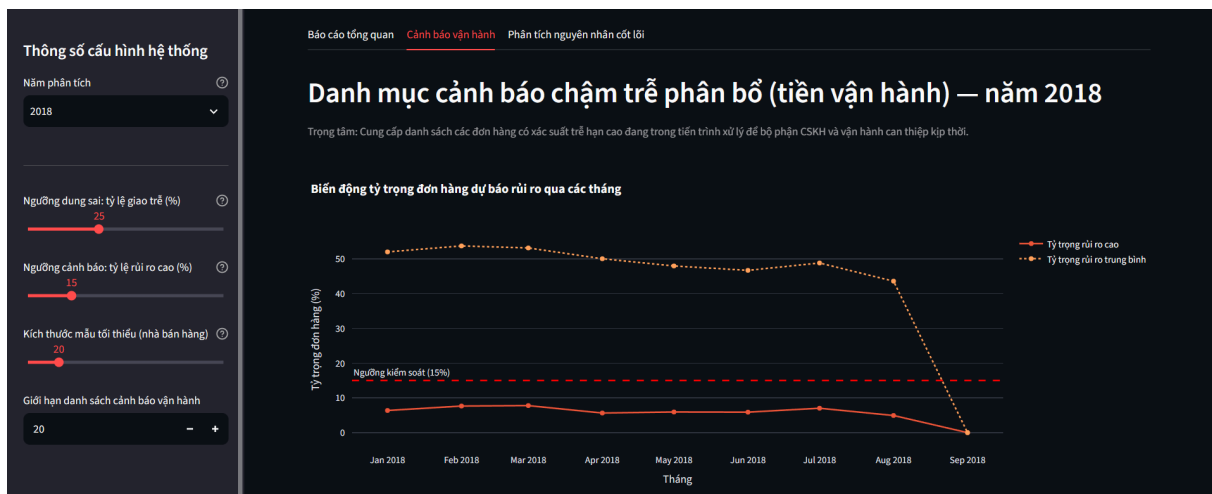


- Extract dữ liệu từ PostgreSQL
- Ghi Bronze
- Chạy notebook Silver và Gold thông qua Job của databricks



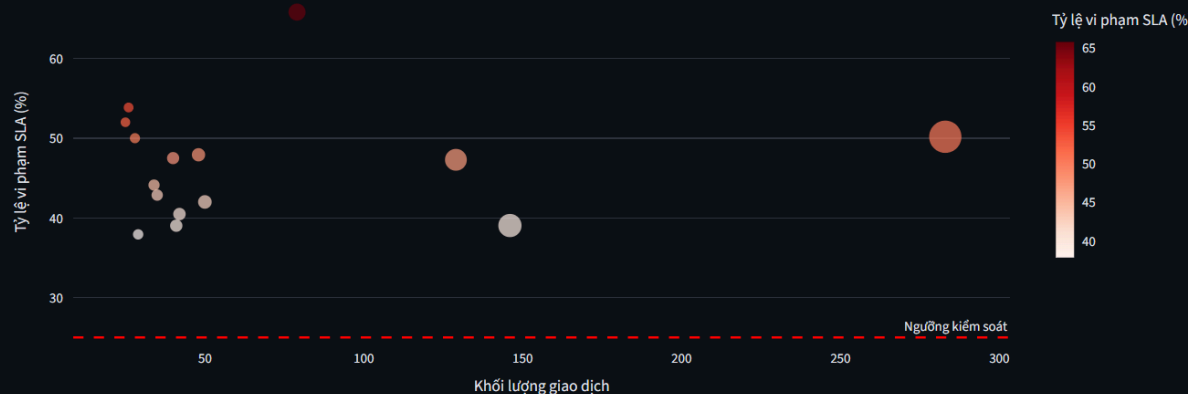
- Cung cấp dữ liệu phục vụ Streamlit Dashboard



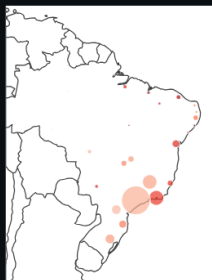


2. Ma trận đánh giá năng lực nhà bán hàng

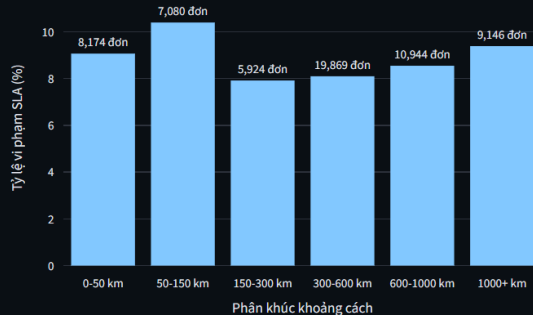
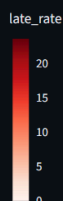
Trọng tâm: Khoanh vùng các đối tác cung cấp có rủi ro cao để áp dụng chính sách chế tài hoặc tái đào tạo (yêu cầu ngưỡng mẫu: >20 đơn).



Tỷ lệ vi phạm SLA phân bố theo vùng (bubble size = quy mô đơn)



Sự phụ thuộc của tỷ lệ vi phạm vào khoảng cách địa lý



Các điểm nhấn phân tích (Key insights)

- Phân khúc khoảng cách xa nhất (1000+ km) ghi nhận tỷ lệ vi phạm 9.39%. Khuyến nghị: Điều chỉnh tăng thời gian dự kiến giao hàng (EDD) cho phân khúc này trên hệ thống front-end.

5. Thiết kế lưu trữ & mô hình dữ liệu

5.1 Bronze Layer

- Lưu dữ liệu gốc trích xuất từ PostgreSQL
- Định dạng Parquet
- Phục vụ truy vết và tái xử lý

5.2 Silver Layer

- Chuẩn hóa kiểu dữ liệu
- Xử lý giá trị thiếu/không hợp lệ
- Loại bỏ trùng lặp
- Kiểm tra chất lượng dữ liệu

5.3 Gold Layer

- Tổng hợp dữ liệu phục vụ phân tích
- Tối ưu cho truy vấn và dashboard

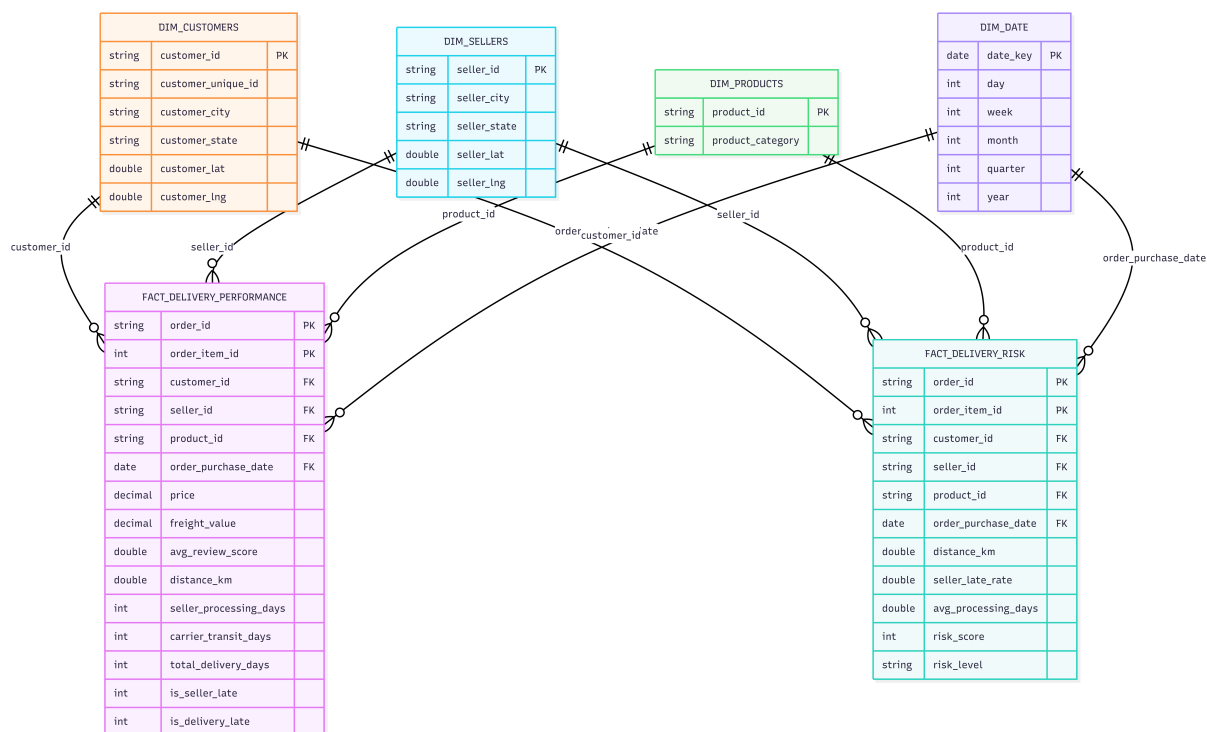
5.4 Data Warehouse

Sau chuyển đổi, dữ liệu Gold được lưu trong Data Warehouse với mục tiêu:

- Tối ưu hiệu năng truy vấn
- Đơn giản hóa phân tích
- Hỗ trợ báo cáo và trực quan hóa

5.5 Galaxy Schema

Data Warehouse được thiết kế theo **Galaxy Schema (Fact Constellation)** với nhiều Fact Table dùng chung các Dimension, phù hợp cho nhiều bài toán phân tích khác nhau.



Trong dự án này, hai Fact Table gồm:

- **fact_delivery_performance**: Phân tích hiệu suất giao hàng.
- **fact_delivery_risk**: Đánh giá và theo dõi rủi ro giao hàng trễ.

Hai Fact sử dụng chung các Dimension như khách hàng, người bán, sản phẩm và thời gian, giúp:

- Tái sử dụng dữ liệu chiều (Dimension).
- Giảm trùng lặp dữ liệu.
- Tăng hiệu năng truy vấn cho nhiều bài toán phân tích.
- Thuận tiện mở rộng Data Warehouse trong tương lai.

6. Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Ngôn ngữ	Python	Xây dựng ETL pipeline & sinh dữ liệu mô phỏng
Database	PostgreSQL	Lưu dữ liệu nguồn (OLTP) & metadata pipeline
Orchestration	Apache Airflow	Lập lịch và điều phối pipeline

Thành phần	Công nghệ	Vai trò
Processing	Databricks (PySpark, SQL)	Xử lý dữ liệu Silver & Gold
Data Lake	Databricks Volume	Lưu trữ dữ liệu Bronze (Parquet)
Data format	Apache Parquet	Định dạng tối ưu cho phân tích
Data warehouse	Databricks SQL	Lưu trữ các bảng Gold phục vụ truy vấn
Dashboard	Streamlit	Trực quan hóa dữ liệu & theo dõi KPI
Data generation	Faker	Sinh dữ liệu giao dịch mô phỏng
Containerization	Docker Compose	Chuẩn hóa môi trường dev/deploy

7. Kết quả & hướng phát triển

7.1 Kết quả

Dự án đã xây dựng được hệ thống dữ liệu end-to-end phục vụ theo dõi và phát hiện sớm rủi ro giao hàng trễ:

- ETL tự động từ PostgreSQL → Databricks (Medallion)
- Full Load + Incremental Load theo watermark
- Metadata & lịch sử chạy pipeline
- Data Warehouse theo Star Schema
- Dashboard Streamlit theo dõi KPI

7.2 Hướng phát triển

- Tích hợp nguồn dữ liệu thời gian thực.
- Bổ sung cơ chế cảnh báo khi phát hiện đơn hàng có nguy cơ giao trễ.
- Xây dựng mô hình Machine Learning dự đoán khả năng giao hàng trễ.
- Mở rộng dashboard với nhiều chỉ số/báo cáo chuyên sâu.
- Triển khai pipeline trên môi trường cloud để tăng khả năng mở rộng và vận hành.